

LAYER 11 OF 13

LOAD BALANCING SCALING

Handling More Users Without Crashing

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 11: Load Balancing & Scaling—what happens when your app goes from ten users to ten thousand in the same hour. When you first launch, your app runs on a single server. It's like a food truck: one window, one cook, works fine when there's a short line. But when a hundred people show up at lunch, that single window becomes a bottleneck. Load balancing is adding more windows. Scaling is hiring more cooks.

You're not going to configure servers by hand. You're going to describe your traffic expectations to your AI coding tool and let it set up the infrastructure. Your job is to understand why your app slows down under pressure, what load balancing actually does (spreads incoming users across multiple copies of your app), and how scaling works (adding more capacity when traffic spikes and shrinking when it's quiet). That's what this certification proves.

This kit walks you through three levels. Tier 1 is for a solo builder who needs their app to survive a busy day without crashing. Tier 2 is for a growing product that needs to handle steady traffic increases automatically. Tier 3 is for platforms running at enterprise scale across multiple regions. Start at Tier 1—you can always come back.

Why It Matters

Here's the brutal truth: your app works perfectly right now because nobody is using it. The moment you get featured on social media, land a big client, or run a successful promotion, hundreds or thousands of people will hit your app at the same time. If you haven't thought about load balancing, your app crashes. Your users see error pages. And they don't come back.

AI coding tools can build you a beautiful app that falls over the moment real traffic arrives. They'll set up a single server, a single database connection, and call it done—because you didn't ask for anything else. This kit teaches you what to ask for so your app stays up when success actually happens. The best time to think about scaling is before you need it.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier

Who It's For

What You Prove

Tier 1

Solo builders shipping apps that need to handle real traffic

You understand why apps crash under load, can describe basic load balancing to AI, and know how to set up auto-scaling so your app survives traffic spikes.

Tier 2

Growing teams with apps serving thousands of daily users

You can manage multi-server deployments, configure connection pooling, set up database read replicas, and implement health-check-based routing.

Tier 3

Platforms with global traffic and high-availability requirements

You can architect multi-region deployments, implement advanced traffic shaping, manage global load balancers, and design for zero-downtime scaling.

TIER 1 CERTIFICATION GOAL

You understand why apps crash under heavy traffic, can describe load balancing and auto-scaling requirements to AI, and can verify that your app handles traffic spikes without going down.

What You Need to Know

You don't need to configure servers. You need to understand a handful of concepts so you can tell AI what you need and verify it's set up correctly.

Why apps crash under load: Your app runs on a server—a computer somewhere in the cloud. That computer has limits: it can only handle so many requests at once. When more people show up than your server can handle, it starts responding slowly, then stops responding at all. That's a crash. Load balancing prevents this by spreading visitors across multiple copies of your app.

Load balancer basics: A load balancer is a traffic cop that sits in front of your app. When a user visits your site, the load balancer decides which copy of your app handles that request. If one copy is busy, it sends the user to a less busy one. Your users never see this—they just experience a fast app. Services like Vercel, Railway, and Fly.io include basic load balancing automatically.

Horizontal vs. vertical scaling: Vertical scaling means making your one server bigger (more memory, faster processor)—like buying a bigger food truck. Horizontal scaling means running multiple copies of your app—like opening more food trucks. Horizontal scaling is almost always better because there's no limit to how many copies you can run, and if one crashes, the others keep going.

Auto-scaling (growing and shrinking automatically): Auto-scaling watches your traffic and adds more copies of your app when things get busy, then removes them when traffic drops. You're not paying for ten servers at 3 AM when nobody's online. Most cloud platforms (Vercel, Fly.io, AWS) offer auto-scaling—you just need to tell AI to set it up.

The vibecoder scaling workflow: Describe your expected traffic to AI → AI configures load balancing and auto-scaling → you test with a load testing tool to simulate heavy traffic → you check that your app stays responsive → iterate until it handles your target load → ship with confidence.

Your Toolkit

Your cloud platform handles most of this. Your job is knowing what to ask for and how to verify it works.

Cloud platform (pick one): Vercel, Fly.io, Railway, Render, or AWS. These platforms run your app and handle load balancing for you. Tell AI which one you're using so it configures things correctly.

Load testing tool: Tools like k6 or Artillery let you simulate hundreds or thousands of users hitting your app at once. This is how you find out if your app survives before real users show up.

Monitoring dashboard: Your cloud platform's built-in metrics show you how many requests your app is handling, response times, and error rates. Check these after running a load test.

Certification Exam Topics

Every exam question is scenario-based. You'll see a situation and need to identify what's wrong or what to do next. Here's what gets tested:

Traffic spikes: Your app was fine yesterday but crashed this morning when a popular blog linked to it. What happened and what should you have set up to prevent it?

Load balancer role: A user complains your app is slow. You check and see one server at 95% capacity while another is at 10%. What's missing?

Scaling direction: Your single server keeps running out of memory during peak hours. Should you make the server bigger or add more servers? Why?

Auto-scaling verification: You told AI to set up auto-scaling. How do you verify it's actually working before real traffic arrives?

Cost awareness: Your app auto-scaled to twenty copies overnight when there were zero users. What went wrong with your scaling configuration?

Session stickiness: A user logs in, adds items to their cart, then gets bounced to a different server and their cart is empty. What's the problem?

Health checks: One of your three app copies has crashed but the load balancer keeps sending traffic to it. What's missing?

Platform selection: You're launching a simple app and want load balancing without managing servers yourself. What kind of platform should you use and why?

Common Pitfalls

These are the mistakes vibecoders make most often with load balancing and scaling. They're easy to make because AI doesn't flag them unless you ask. Spot them now so you don't discover them during your first traffic spike.

Never load-testing before launch. Your app works fine with one user—you. The first time a hundred people show up simultaneously, you'll find every bottleneck the hard way.

Storing user sessions on the server instead of in a shared store like Redis. The moment you add a second server copy, users start losing their login state and cart contents.

Ignoring database connection limits. Your app works fine locally with one connection but crashes in production because twenty app copies are each trying to open fifty connections to the same database.

Setting up auto-scaling without setting a maximum. Without a cap, a traffic bot or a bug in your code can spin up hundreds of server copies and run up a massive cloud bill overnight.

Not setting up health checks. Without them, your load balancer keeps sending traffic to crashed copies of your app, and users see errors while healthy copies sit underused.

Scaling the app servers but forgetting about the database. Your app can handle ten thousand requests per second, but your single database can only handle five hundred. The database becomes the bottleneck and everything slows down.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every “no” is something to work on.

Can you explain in plain language why an app that works for ten users might crash for a thousand?

Do you know the difference between horizontal scaling (more copies) and vertical scaling (bigger server)—and which one to ask AI for?

Have you run a load test on your app to see how it handles simulated traffic?

Can you describe to AI what a load balancer does and ask it to set one up for your deployment?

Do you know what auto-scaling is and how to tell AI to configure it with a minimum, maximum, and scaling trigger?

If a user told you the app was slow, would you know which metrics to check—response time, error rate, server CPU usage?

Can you explain what a health check is and why your load balancer needs one to work properly?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick health check on your load balancing and scaling setup. It checks the same things the certification exam covers.

Review my app's load balancing and scaling setup and check the following. For each one, tell me pass or fail with a specific example:

Load balancing: Is my app configured to distribute traffic across multiple copies, or is everything running on a single instance?

Auto-scaling: Is auto-scaling configured with sensible minimum and maximum limits and a clear trigger (like CPU usage or request count)?

Health checks: Does my load balancer have health check endpoints that automatically remove unhealthy copies from rotation?

Session management: Is user session data stored in a shared store (like Redis) so it works across multiple app copies, or is it stuck on individual servers?

Database connections: Is connection pooling configured so my app doesn't overwhelm the database with too many simultaneous connections?

Database scaling: Are read replicas set up to handle read-heavy traffic, or is my single database handling everything?

Give me an overall score out of 6 and list the top 3 things to fix first.

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 11 certification exam at your target tier.

The best way to prepare: run a load test on your actual app. Simulate a hundred users, then a thousand. Watch what breaks. Check your response times, error rates, and server metrics. Every bottleneck you find and fix is exactly what the exam tests. The goal isn't to memorize terms—it's to prove you can keep your app running when real users show up.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai

© 2026 Matt Murphy .AI. All rights reserved.